

=====

CAPSTONE PROJECT

NOTEBOOK 12: EXPAND 8-GENRE PILOT AND RETRAIN

=====

Purpose:

This notebook expands the current 8-genre pilot subset using

more labelled examples from FMA-Large while preserving the

official split logic.

The goal is to:

1. Rebuild the pilot subset with larger sample caps
2. Keep the same 8 genres used in the validated hybrid system
3. Save the expanded pilot metadata

4. Reuse the same downstream structured, CNN, and hybrid

notebooks on a larger but still controlled task

=====

In [1]:

```
# =====  
# 1. IMPORT LIBRARIES  
# =====  
  
import os  
import random  
import numpy as np  
import pandas as pd  
  
SEED = 42  
random.seed(SEED)  
np.random.seed(SEED)  
  
print("Seeds set to:", SEED)
```

Seeds set to: 42

In [2]:

```
# =====  
# 2. LOAD TRACK METADATA  
# =====  
  
tracks = pd.read_csv(  
    "../data/raw/metadata/tracks.csv",  
    header=[0, 1],  
    index_col=0  
)  
  
tracks.index = tracks.index.astype(int)  
  
print("Tracks shape:", tracks.shape)  
display(tracks.head())
```

Tracks shape: (106574, 52)

	comments	date_created	date_released	engineer	favorites	id	information	listens
track_id								
2	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
3	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
5	0	2008-11-26 01:44:45	2009-01-05 00:00:00	NaN	4	1		6073
10	0	2008-11-26 01:45:08	2008-02-06 00:00:00	NaN	4	6	NaN	47632
20	0	2008-11-26 01:45:05	2009-01-06 00:00:00	NaN	2	4	"spiritual songs" from Nicky Cook	2710

5 rows × 52 columns

In [3]:

```
# =====
# 3. DEFINE AUDIO PATH FUNCTION
# =====

def get_audio_path(track_id, base_dir="../../data/raw/audio/fma_large"):
    track_id_str = f"{int(track_id):06d}"
    folder = track_id_str[:3]
    return os.path.join(base_dir, folder, f"{track_id_str}.mp3")
```

In [4]:

```
# =====
# 4. FILTER TO LABELLED TRACKS WITH AUDIO PRESENT
# =====

large_tracks = tracks[tracks[("set", "subset")] == "large"].copy()
large_tracks = large_tracks[large_tracks[("track", "genre_top")].notna()].copy()

large_tracks["audio_path"] = [get_audio_path(idx) for idx in large_tracks.index]
```

```

large_tracks["audio_exists"] = large_tracks["audio_path"].apply(os.path.exists)

large_tracks = large_tracks[large_tracks["audio_exists"] == True].copy()

print("Large labelled tracks with audio:", large_tracks.shape)
print("\nGenre counts:")
print(large_tracks[["track", "genre_top"]].value_counts().head(20))

```

Large labelled tracks with audio: (24598, 54)

Genre counts:

(track, genre_top)

Experimental	8357
Rock	7079
Electronic	3058
Hip-Hop	1351
Folk	1284
Pop	1146
Instrumental	729
Classical	611
International	371
Spoken	305
Jazz	187
Old-Time / Historic	44
Blues	36
Soul-RnB	21
Country	16
Easy Listening	3

Name: count, dtype: int64

In [5]:

```

# =====
# 5. KEEP THE SAME 8 GENRES AS THE CURRENT HYBRID SYSTEM
# =====

pilot_genres = [
    "Classical",
    "Electronic",
    "Experimental",
    "Folk",
    "Hip-Hop",
    "Instrumental",
    "Pop",
    "Rock"
]

pilot_tracks = large_tracks[
    large_tracks[["track", "genre_top"]].isin(pilot_genres)
].copy()

print("Pilot candidate shape:", pilot_tracks.shape)
print("\nPilot genre counts:")
print(pilot_tracks[["track", "genre_top"]].value_counts())

```

Pilot candidate shape: (23615, 54)

Pilot genre counts:
(track, genre_top)
Experimental 8357
Rock 7079
Electronic 3058
Hip-Hop 1351
Folk 1284
Pop 1146
Instrumental 729
Classical 611
Name: count, dtype: int64

In [6]:

```
# =====  
# 6. CHECK AVAILABLE COUNTS BY SPLIT AND GENRE  
# =====  
  
split_genre_counts = (  
    pilot_tracks  
    .groupby(["set", "split"], ("track", "genre_top"))  
    .size()  
    .unstack(fill_value=0)  
)  
  
print("Available counts by split and genre:")  
display(split_genre_counts)
```

Available counts by split and genre:

(track, genre_top)	Classical	Electronic	Experimental	Folk	Hip- Hop	Instrumental	Pop	Rock
(set, split)								
test	25	207	860	147	103	135	85	753
training	574	2612	6756	1060	1149	534	870	5713
validation	12	239	741	77	99	60	191	613

In [7]:

```
# =====  
# 7. CHOOSE EXPANDED SAMPLE CAPS  
# =====  
# We choose conservative caps so that every genre can support  
# the same train/validation/test counts.  
#  
# Start with these suggested targets. If any genre cannot support  
# them, the code below will show it and we can lower them.  
  
TRAIN_CAP = 300  
VAL_CAP = 50
```

```

TEST_CAP = 50

print("Requested caps:")
print("TRAIN_CAP =", TRAIN_CAP)
print("VAL_CAP   =", VAL_CAP)
print("TEST_CAP  =", TEST_CAP)

for genre in pilot_genres:
    train_available = split_genre_counts.loc["training", genre] if "training" in
split_genre_counts.index else 0
    val_available = split_genre_counts.loc["validation", genre] if "validation" in
split_genre_counts.index else 0
    test_available = split_genre_counts.loc["test", genre] if "test" in
split_genre_counts.index else 0

    print(f"{genre}: train={train_available}, val={val_available}, test=
{test_available}")

```

Requested caps:
 TRAIN_CAP = 300
 VAL_CAP = 50
 TEST_CAP = 50
 Classical: train=574, val=12, test=25
 Electronic: train=2612, val=239, test=207
 Experimental: train=6756, val=741, test=860
 Folk: train=1060, val=77, test=147
 Hip-Hop: train=1149, val=99, test=103
 Instrumental: train=534, val=60, test=135
 Pop: train=870, val=191, test=85
 Rock: train=5713, val=613, test=753

In [8]:

```

# =====
# 8. BUILD EXPANDED BALANCED PILOT SUBSET
# =====

expanded_parts = []

for genre in pilot_genres:
    genre_df = pilot_tracks[pilot_tracks[("track", "genre_top")] == genre].copy()

    train_df = genre_df[genre_df[("set", "split")] == "training"].sample(
        n=min(TRAIN_CAP, len(genre_df[genre_df[("set", "split")] == "training"])),
        random_state=SEED
    )

    val_df = genre_df[genre_df[("set", "split")] == "validation"].sample(
        n=min(VAL_CAP, len(genre_df[genre_df[("set", "split")] == "validation"])),
        random_state=SEED
    )

    test_df = genre_df[genre_df[("set", "split")] == "test"].sample(
        n=min(TEST_CAP, len(genre_df[genre_df[("set", "split")] == "test"])),
        random_state=SEED
    )

```

```

expanded_parts.append(train_df)
expanded_parts.append(val_df)
expanded_parts.append(test_df)

expanded_pilot = pd.concat(expanded_parts).sort_index().copy()

print("Expanded pilot shape:", expanded_pilot.shape)
print("\nExpanded genre counts:")
print(expanded_pilot[("track", "genre_top")].value_counts())

print("\nExpanded split counts:")
print(expanded_pilot[("set", "split")].value_counts())

```

Expanded pilot shape: (3137, 54)

Expanded genre counts:

(track, genre_top)

Rock 400

Folk 400

Experimental 400

Hip-Hop 400

Electronic 400

Pop 400

Instrumental 400

Classical 337

Name: count, dtype: int64

Expanded split counts:

(set, split)

training 2400

test 375

validation 362

Name: count, dtype: int64

In [9]:

```

# =====
# 9. CREATE CLEAN EXPORT DATAFRAME
# =====

expanded_export = pd.DataFrame({
    "track_id": expanded_pilot.index.astype(int),
    "genre_top": expanded_pilot[("track", "genre_top")].astype(str).values,
    "split": expanded_pilot[("set", "split")].astype(str).values,
    "audio_path": expanded_pilot["audio_path"].astype(str).values,
    "audio_exists": expanded_pilot["audio_exists"].astype(bool).values
})

print("Expanded export shape:", expanded_export.shape)
display(expanded_export.head())

```

Expanded export shape: (3137, 5)

	track_id	genre_top	split	audio_path	audio_exists
0	171	Rock	training	../data/raw/audio/fma_large\000\000171.mp3	True
1	178	Rock	training	../data/raw/audio/fma_large\000\000178.mp3	True
2	189	Folk	training	../data/raw/audio/fma_large\000\000189.mp3	True
3	191	Folk	training	../data/raw/audio/fma_large\000\000191.mp3	True
4	205	Folk	training	../data/raw/audio/fma_large\000\000205.mp3	True

In [10]:

```
# =====
# 10. SAVE EXPANDED PILOT METADATA
# =====

os.makedirs("../data/processed", exist_ok=True)

expanded_export.to_csv(
    "../data/processed/audio_large_pilot_metadata_expanded.csv",
    index=False
)

print("Saved expanded pilot metadata to:")
print("../data/processed/audio_large_pilot_metadata_expanded.csv")
```

Saved expanded pilot metadata to:

../data/processed/audio_large_pilot_metadata_expanded.csv

In [11]:

```
# =====
# 11. INTERPRETATION NOTES
# =====

print("1. This notebook rebuilds the same 8-genre task using more examples per class.")
print("2. The official train/validation/test split is still preserved.")
print("3. The saved expanded pilot file can now be used in the audio, structured, and hybrid notebooks.")
print("4. This is the safest next step for improving accuracy before increasing genre coverage.")
```

1. This notebook rebuilds the same 8-genre task using more examples per class.
2. The official train/validation/test split is still preserved.
3. The saved expanded pilot file can now be used in the audio, structured, and hybrid notebooks.
4. This is the safest next step for improving accuracy before increasing genre coverage.